

Chương 3 đã trình bày nền tảng lý thuyết và các công nghệ được lựa chọn cho từng tầng trong kiến trúc hệ thống. Trên cơ sở đó, Chương 4 trình bày chi tiết quá trình thiết kế, triển khai và đánh giá hệ thống. Nội dung chương bao gồm thiết kế kiến trúc (phần 0.1), thiết kế chi tiết (phần 0.2), xây dựng ứng dụng (phần 0.3), kiểm thử (phần 0.4), và triển khai (phần 0.5).

0.1 Thiết kế kiến trúc

0.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được thiết kế theo kiến trúc bốn tầng, bao gồm tầng cảm biến (Sensor Layer), tầng trung chuyển (Gateway Layer), tầng máy chủ (Server Layer) và tầng hiển thị (Presentation Layer). Kiến trúc bốn tầng được lựa chọn vì ba lý do chính. Thứ nhất, mỗi tầng có trách nhiệm rõ ràng và có thể phát triển độc lập: tầng cảm biến thu thập dữ liệu môi trường, tầng trung chuyển chuyển tiếp dữ liệu từ mạng Long Range (LoRa) sang Internet, tầng máy chủ xử lý nghiệp vụ và lưu trữ, tầng hiển thị cung cấp giao diện cho người dùng. Thứ hai, kiến trúc này cho phép mở rộng theo chiều ngang tại từng tầng — ví dụ thêm Sensor Node mới mà không thay đổi phần mềm Backend. Thứ ba, độ phức tạp phù hợp với quy mô hệ thống: triển khai trên một máy chủ VPS duy nhất, không yêu cầu kiến trúc Microservice.

Tầng cảm biến gồm các Sensor Node, mỗi node tích hợp vi điều khiển ESP32, ba module cảm biến (PMS7003, CCS811, AHT10) và module LoRa AS32-TTL-100. Firmware sử dụng FreeRTOS để quản lý đồng thời bốn tác vụ: đọc cảm biến, gửi dữ liệu LoRa, hiển thị OLED và đo pin. Dữ liệu được đóng gói thành gói tin nhị phân 18 byte và phát qua LoRa 433 MHz.

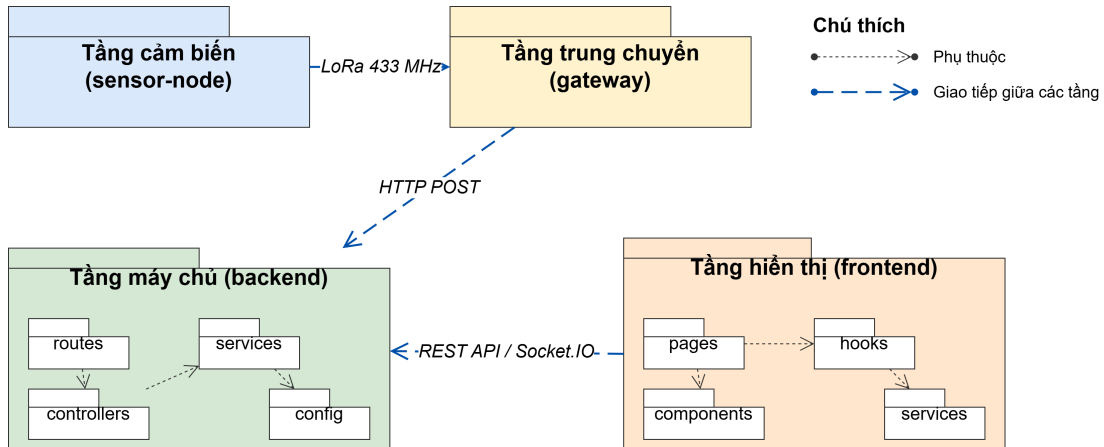
Tầng trung chuyển gồm các Gateway, mỗi Gateway tích hợp ESP32 với module LoRa và kết nối WiFi. Gateway nhận gói tin LoRa, lưu vào bộ đệm vòng (ring buffer), sau đó gửi lên tầng máy chủ dưới dạng HTTP POST (JSON batch). Ở tầng này, hệ thống áp dụng mô hình Hub-Spoke: nhiều Sensor Node (spoke) gửi dữ liệu đến một Gateway (hub) trung tâm, cho phép một Gateway phục vụ nhiều Sensor Node trong phạm vi phủ sóng mà không cần định tuyến phức tạp.

Tầng máy chủ bao gồm Backend Server (Node.js/Express) và cơ sở dữ liệu (PostgreSQL với TimescaleDB và PostGIS), được đóng gói trong Docker container. Backend Server được tổ chức theo mô hình phân lớp gồm ba lớp: lớp Controller tiếp nhận HTTP request, lớp Service chứa logic nghiệp vụ, và lớp Data Access tương tác với cơ sở dữ liệu thông qua Prisma ORM. Ngoài ra, tầng máy chủ còn có Nginx đóng vai trò reverse proxy và phục vụ tệp tĩnh, cùng các tác vụ nền (cron job) kiểm tra thiết bị offline và dọn dẹp dữ liệu cũ.

Tầng hiển thị là ứng dụng web SPA xây dựng bằng React 19 và Vite 8, giao

tiếp với tầng máy chủ qua REST API (HTTP/HTTPS) và Socket.IO (WebSocket). Tầng này cung cấp hai nhóm giao diện: dashboard cho người dùng (bản đồ AQI, biểu đồ lịch sử, xếp hạng) và trang quản trị cho quản trị viên (quản lý thiết bị, cảnh báo, cấu hình, xuất dữ liệu).

0.1.2 Thiết kế tổng quan



Hình 0.1: Biểu đồ gói UML tổng quan của hệ thống

Hệ thống được tổ chức thành bốn nhóm gói chính tương ứng với bốn tầng kiến trúc, mỗi nhóm chứa các gói con với trách nhiệm cụ thể.

Nhóm gói Tầng cảm biến (sensor-node) chứa các module: `drivers` (giao tiếp cảm biến PMS7003, CCS811, AHT10, màn hình OLED, module LoRa), `core` (quản lý cấu hình NVS, provisioning qua WiFi AP, vòng lặp chính), và `config` (định nghĩa chân GPIO, tham số LoRa). Hệ thống có ba biến thể firmware cho các loại board ESP32 khác nhau.

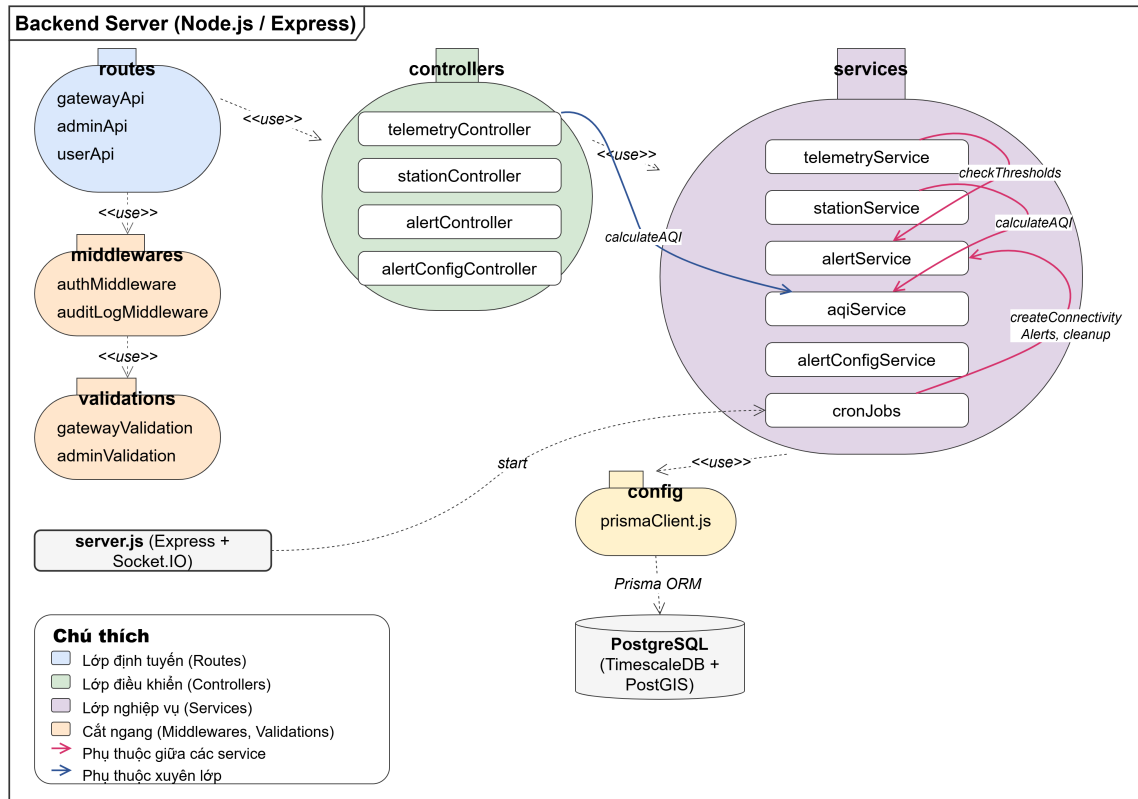
Nhóm gói Tầng trung chuyển (gateway) chứa các module: `drivers` (giao tiếp LoRa), `core` (bộ đệm vòng, gửi HTTP batch lên tầng máy chủ), và `config` (cấu hình WiFi, địa chỉ Backend). Nhóm gói này hoạt động độc lập, không phụ thuộc vào nhóm gói tầng cảm biến.

Nhóm gói Tầng máy chủ (backend) gồm sáu gói: `routes` (định nghĩa endpoint API), `controllers` (tiếp nhận request, trả response), `services` (logic nghiệp vụ), `middlewares` (xác thực JWT, xử lý lỗi), `validations` (kiểm tra dữ liệu đầu vào), và `config` (kết nối Prisma Client). Luồng phụ thuộc tuân thủ nguyên tắc một chiều: `routes` → `controllers` → `services` → `config` (Prisma), đảm bảo gói tầng dưới không phụ thuộc gói tầng trên.

Nhóm gói Tầng hiển thị (frontend) gồm bảy gói: `pages` (các trang giao diện), `components` (thành phần tái sử dụng), `hooks` (custom React hooks), `services` (gọi API), `stores` (quản lý trạng thái Zustand), `layouts` (bố cục trang),

và i18n (đa ngôn ngữ). Gói pages phụ thuộc components và hooks; gói hooks phụ thuộc services; gói services gọi REST API đến tầng máy chủ.

0.1.3 Mô tả luồng dữ liệu giữa các gói



Hình 0.2: Biểu đồ quan hệ module cho backend services

Hình 0.2 mô tả quan hệ phụ thuộc giữa các module trên Backend. Hai nhóm module quan trọng nhất là nhóm xử lý dữ liệu telemetry và nhóm hiển thị dashboard.

Nhóm xử lý dữ liệu telemetry bao gồm bốn module: telemetryController tiếp nhận request, telemetryValidation xác thực đầu vào, telemetryService xử lý nghiệp vụ (deduplication, lưu trữ, kiểm tra cảnh báo), và aqiService tính chỉ số AQI. Luồng xử lý chi tiết được trình bày qua ba biểu đồ trình tự ở phần 0.2.4.

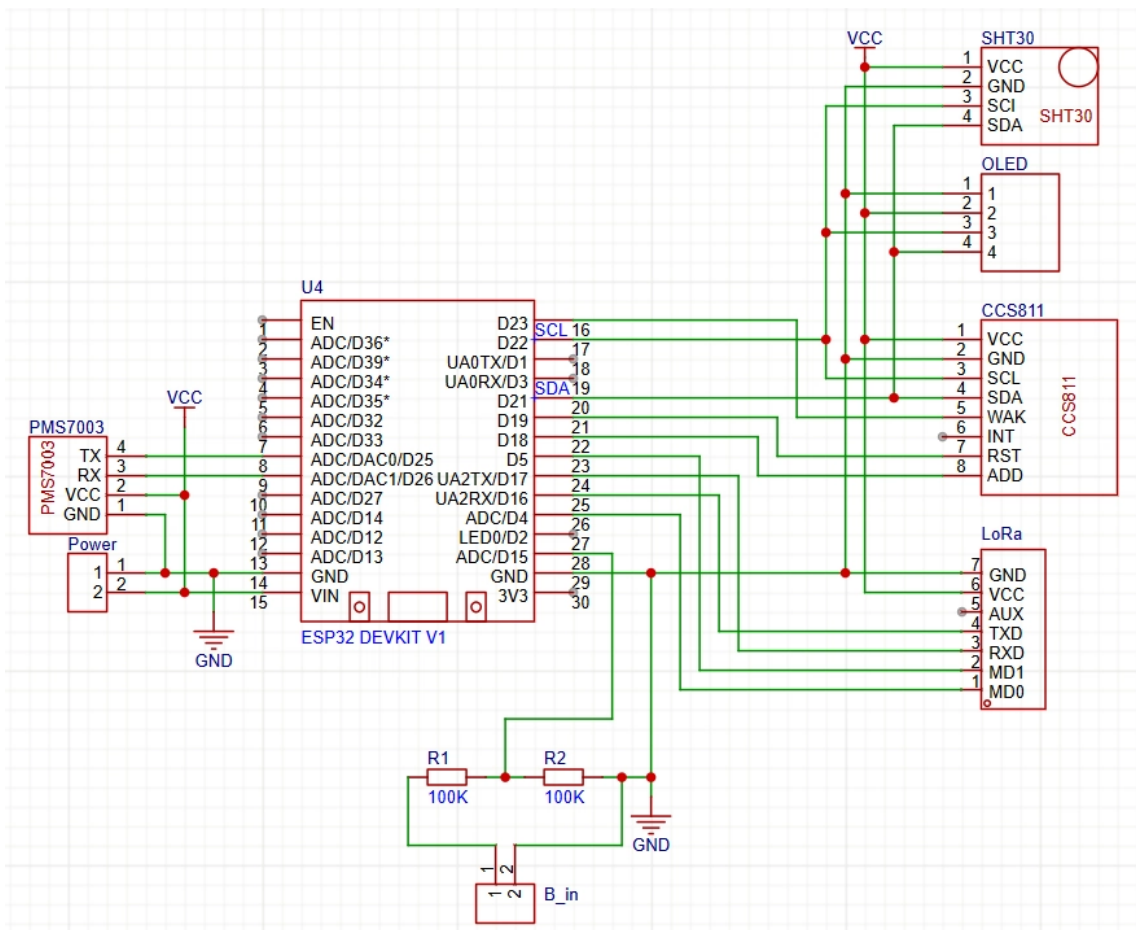
Nhóm hiển thị dashboard trên Frontend sử dụng hook useDashboard (gọi API lấy dữ liệu ban đầu) và hook useSocket (nhận cập nhật thời gian thực qua Socket.IO). Chi tiết luồng end-to-end từ truy cập Dashboard đến cập nhật realtime được mô tả ở Hình 0.6.

0.2 Thiết kế chi tiết

0.2.1 Thiết kế phần cứng

Phần này trình bày thiết kế mạch kết nối phần cứng cho Sensor Node và Gateway.

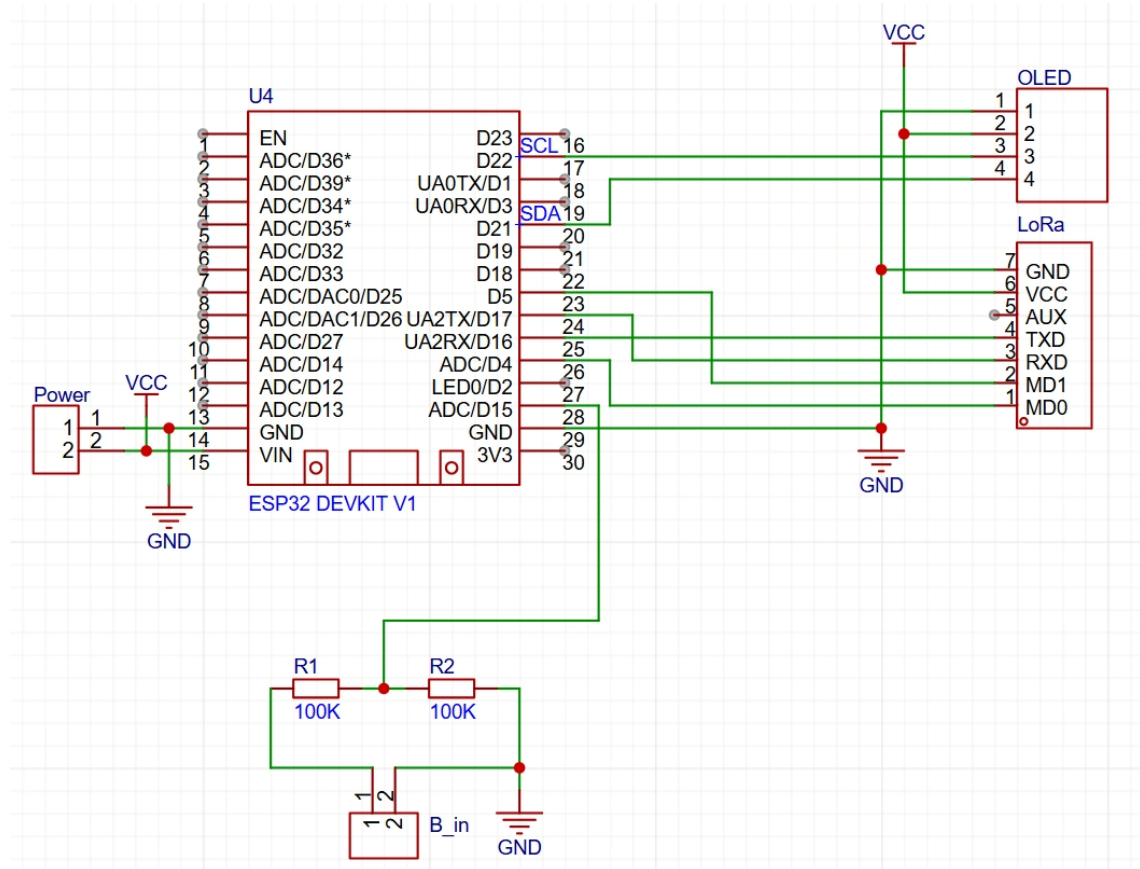
Sensor Node sử dụng board ESP32 DevKit V1 (30 chân) làm trung tâm, kết nối với bốn module ngoại vi qua ba giao tiếp khác nhau. Module LoRa AS32-TTL-100 giao tiếp qua UART1 (RX: GPIO16, TX: GPIO17) với hai chân điều khiển chế độ (MD0: GPIO4, MD1: GPIO5). Module cảm biến bụi PMS7003 giao tiếp qua UART2 (RX: GPIO25, TX: GPIO26). Module CCS811 và AHT10 chia sẻ chung bus I2C (SDA: GPIO21, SCL: GPIO22) với địa chỉ lần lượt là 0x5A và 0x38; CCS811 có thêm chân WAK (GPIO23) để đánh thức từ chế độ ngủ. Màn hình OLED SSD1306 cũng dùng chung bus I2C trên, địa chỉ 0x3C. Pin 18650 được đo qua mạch phân áp (voltage divider R1=R2=100K Ω) nối vào GPIO15 (ADC). Nút BOOT (GPIO0) được tận dụng làm nút factory reset: giữ 5 giây để xóa cấu hình và vào chế độ provisioning.



Hình 0.3: Sơ đồ mạch Sensor Node

Gateway sử dụng cùng board ESP32 DevKit V1, nhưng chỉ kết nối một module

ngoại vi: module LoRa AS32-TTL-100 qua UART1 (cùng cấu hình chân với Sensor Node). Gateway không cần cảm biến hay màn hình OLED, do chức năng duy nhất là nhận gói LoRa và chuyển tiếp lên tầng máy chủ qua WiFi. Thiết kế tối giản này giảm chi phí linh kiện và giảm tiêu thụ năng lượng cho Gateway.



Hình 0.4: Sơ đồ mạch Gateway

0.2.2 Thiết kế firmware

Phần này trình bày thiết kế firmware cho Sensor Node và Gateway.

Firmware Sensor Node sử dụng FreeRTOS với bốn tác vụ chạy song song trên hai nhân của ESP32. Tác vụ `sensor_task` (ưu tiên 2, Core 0, stack 4096 byte) đọc dữ liệu từ ba cảm biến theo chu kỳ 15 giây, thực hiện đến 3 lần retry khi đọc lỗi, sau đó đẩy dữ liệu vào hàng đợi FreeRTOS (queue depth = 3). Tác vụ `lora_task` (ưu tiên 3 — cao nhất, Core 1) nhận dữ liệu từ hàng đợi, đóng gói thành struct `SensorPayload` và phát qua module LoRa. Tác vụ `battery_task` (ưu tiên 1, Core 0) đọc pin mỗi 30 giây qua ADC với 16 mẫu lấy trung bình. Tác vụ `watch-dog_task` (ưu tiên 0, Core 0) giám sát hoạt động: nếu hệ thống không gửi dữ liệu trong 60 giây, tự động reset ESP32.

Cấu trúc gói tin LoRa được định nghĩa dưới dạng struct C packed (không có padding), tổng kích thước 18 byte. Gói tin gồm: `nodeId` (1 byte, ID node 1–255),

`pktType` (1 byte, phân biệt gói dữ liệu/heartbeat/lỗi), `msgId` (1 byte, bộ đếm tuần hoàn 0–255 phục vụ deduplication), bảy trường dữ liệu cảm biến (PM1, PM2.5, PM10 mỗi trường 2 byte nhân 10, CO₂ và TVOC mỗi trường 2 byte, nhiệt độ 2 byte có dấu nhân 10, độ ẩm 2 byte nhân 10), và `battery` (1 byte, 0–100%). Các giá trị dạng số thực được nhân 10 và lưu dạng số nguyên để tránh sử dụng floating-point trong truyền thông, giảm kích thước gói tin. Giá trị sentinel (0xFFFF cho unsigned, 0x7FFF cho signed) đánh dấu cảm biến bị lỗi.

Cơ chế provisioning cho phép cấu hình Node ID qua giao diện web. Khi chưa có cấu hình (hoặc sau factory reset), Sensor Node khởi động WiFi AP với SSID “AirQuality-SN-Setup”, phục vụ trang web captive portal tại địa chỉ 192.168.4.1. Người dùng nhập Node ID, firmware lưu vào bộ nhớ NVS (Non-Volatile Storage) và khởi động lại ở chế độ hoạt động bình thường.

Firmware Gateway sử dụng kiến trúc superloop (không dùng FreeRTOS) do chỉ thực hiện hai tác vụ tuần tự: nhận gói tin LoRa từ UART và tích lũy vào bộ đệm vòng (ring buffer). Khi đủ số lượng gói hoặc hết chu kỳ thời gian, Gateway serialize bộ đệm thành JSON array và gửi lên tầng máy chủ qua HTTP POST. Gateway cũng hỗ trợ cơ chế self-provisioning: tự đăng ký với Backend Server khi kết nối lần đầu.

0.2.3 Thiết kế giao diện

Giao diện hệ thống được thiết kế cho trình duyệt web, hỗ trợ responsive trên nhiều kích thước màn hình từ máy tính (độ phân giải tối thiểu 1280×720) đến thiết bị di động (tối thiểu 375×667). Hệ thống hỗ trợ hai ngôn ngữ (Tiếng Việt và Tiếng Anh) thông qua thư viện `react-i18next`.

Giao diện được tổ chức thành hai nhóm layout riêng biệt. **UserLayout** dành cho người dùng, gồm thanh điều hướng ngang (navbar) ở trên cùng với các liên kết đến bốn trang: Dashboard, Lịch sử, Bản đồ và Xếp hạng. **AdminLayout** dành cho quản trị viên, gồm thanh điều hướng dọc (sidebar) bên trái có thể thu gọn, chứa các mục: Dashboard, Gateways, Sensor Nodes, Cảnh báo, Cấu hình, Người dùng, Xuất dữ liệu và Nhật ký.

Hệ thống sử dụng bộ thành phần giao diện tái sử dụng (component library) tự xây dựng, bao gồm 28 component: Button, Modal, Badge, Input, Select, Table, Pagination, Card, Tooltip và các component chuyên biệt như AQIGauge, Metric-Card, HealthAdvisory. Tất cả component tuân thủ thiết kế nhất quán: dark theme với bảng màu chính là xanh dương đậm, các trạng thái tương tác (hover, focus, disabled) được định nghĩa thống nhất, và hiệu ứng chuyển đổi mượt mà (transition 200ms).

0.2.4 Thiết kế module Backend

Phần này trình bày thiết kế chi tiết cho bốn module quan trọng nhất trên Backend Server.

Module telemetryService chịu trách nhiệm xử lý dữ liệu đo lường từ Gateway. Module này cung cấp hai hàm chính: `processTelemetry(gateway_id, data)` xử lý gói dữ liệu batch và `processHeartbeat(gateway_id)` xử lý tín hiệu heartbeat. Hàm `processTelemetry` thực hiện tuần tự ba bước: lọc trùng lặp qua module `dedupCache`, thực thi transaction cập nhật trạng thái thiết bị và lưu dữ liệu đo lường, sau đó kiểm tra ngưỡng cảnh báo ngoài transaction.

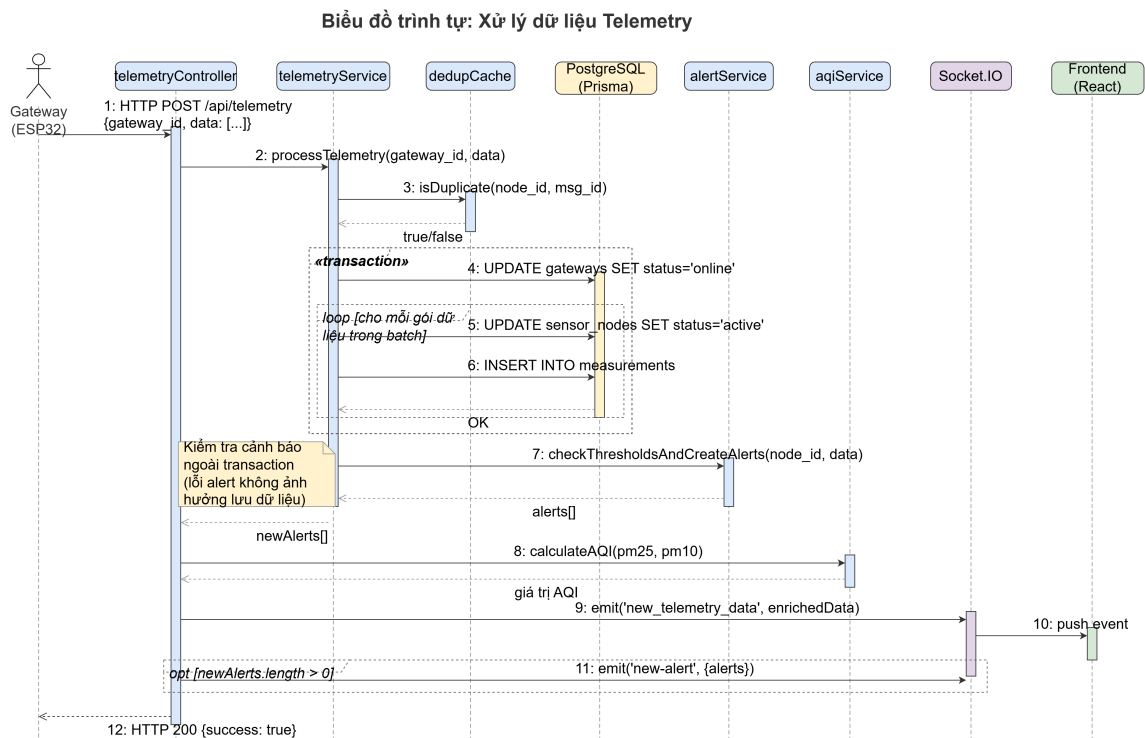
Module aqiService triển khai thuật toán tính Chỉ số chất lượng không khí (Air Quality Index – AQI) theo chuẩn Cơ quan Bảo vệ Môi trường Hoa Kỳ (United States Environmental Protection Agency – US EPA) đã trình bày ở Chương 3. Module cung cấp năm hàm: `calculateSubAQI` tính AQI thành phần cho một chất ô nhiễm, `calculateAQI` tính AQI tổng hợp (giá trị lớn nhất giữa PM2.5 và PM10), `getAQIInfo` ánh xạ giá trị AQI sang mức và mã màu, `getCO2Info` và `getTVOCInfo` đánh giá mức CO₂ và TVOC. Bảng breakpoint được lưu trữ dưới dạng mảng hằng số trong mã nguồn, tương ứng chính xác với Bảng ?? và Bảng ?? ở Chương 3.

Module alertService quản lý toàn bộ vòng đời cảnh báo. Module cung cấp sáu hàm: `getAlerts` truy vấn danh sách cảnh báo với hỗ trợ lọc và phân trang, `acknowledgeAlert` xác nhận xử lý, `deleteAlert` xóa cảnh báo, `checkThresholdsAndCreateAlerts` kiểm tra năm thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ) so với ngưỡng cấu hình và tạo cảnh báo khi vượt ngưỡng, `createConnectivityAlerts` tạo cảnh báo mất kết nối cho thiết bị offline, và `cleanupOldAlerts` xóa cảnh báo quá 30 ngày. Cơ chế cooldown 15 phút ngăn chặn tạo cảnh báo trùng lặp cho cùng thiết bị và cùng thông số trong khoảng thời gian ngắn.

Module stationService cung cấp dữ liệu cho các trang người dùng. Hàm `getNearestStation` sử dụng PostGIS để truy vấn trạm đo gần nhất dựa trên toạ độ GPS của người dùng thông qua hàm `ST_Distance`. Hàm `getStationHistory` truy vấn dữ liệu lịch sử từ Continuous Aggregate `hourly_measurements` cho chế độ xem theo giờ, hoặc từ bảng `measurements` gốc cho chế độ xem theo ngày.

Để minh hoạ luồng tương tác giữa các thành phần, phần tiếp theo trình bày ba biểu đồ trình tự (sequence diagram) cho ba use case quan trọng nhất của hệ thống.

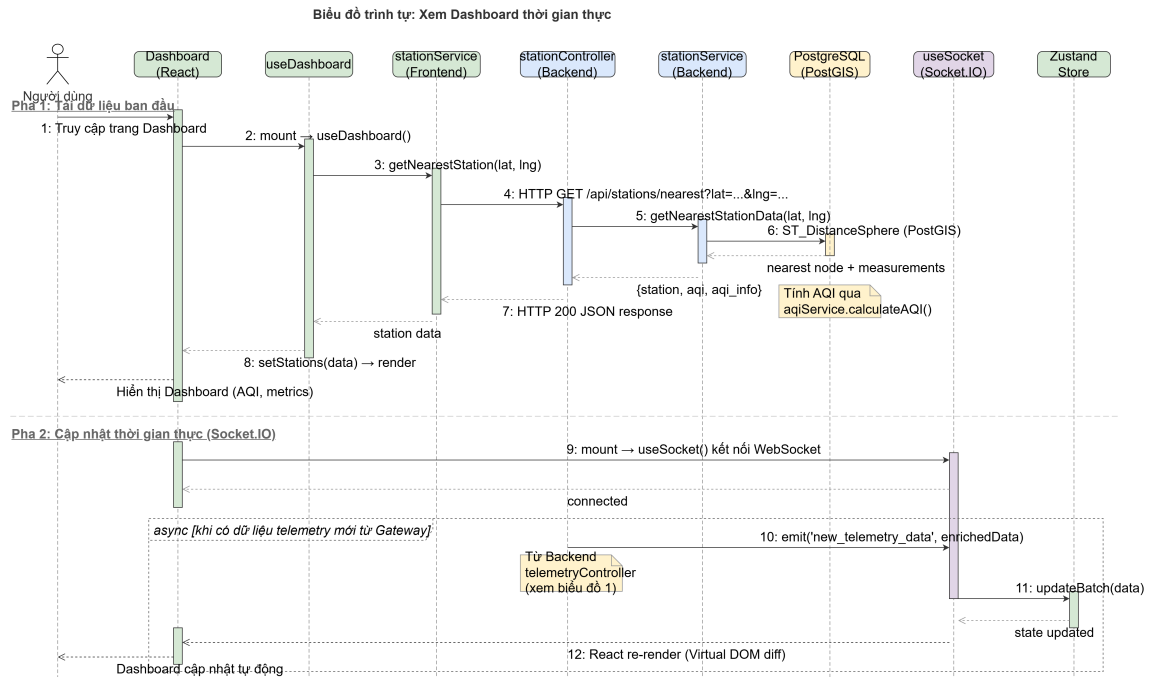
Biểu đồ trình tự 1: Xử lý dữ liệu Telemetry. Hình 0.5 mô tả luồng xử lý khi Gateway gửi dữ liệu đo lường lên Backend — đây là luồng cốt lõi của hệ thống IoT. Gateway gửi HTTP POST chứa batch dữ liệu đến `telemetryController`. Controller chuyển tiếp cho `telemetryService`, module này thực hiện ba bước tuần tự: (i) lọc gói tin trùng lặp qua `dedupCache` (xử lý trường hợp nhiều Gateway cùng nhận một gói LoRa), (ii) thực thi database transaction để cập nhật trạng thái Gateway, cập nhật trạng thái Sensor Node và lưu dữ liệu đo lường vào bảng `measurements`, (iii) kiểm tra ngưỡng cảnh báo qua `alertService` — bước này chạy ngoài transaction để lỗi cảnh báo không ảnh hưởng đến luồng lưu dữ liệu chính. Sau đó, Controller tính chỉ số AQI qua `aqiService` và phát sự kiện `new_telemetry_data` qua Socket.IO đến Frontend. Nếu có cảnh báo mới, sự kiện `new-alert` cũng được phát để thông báo cho quản trị viên.



Hình 0.5: Biểu đồ trình tự xử lý dữ liệu Telemetry

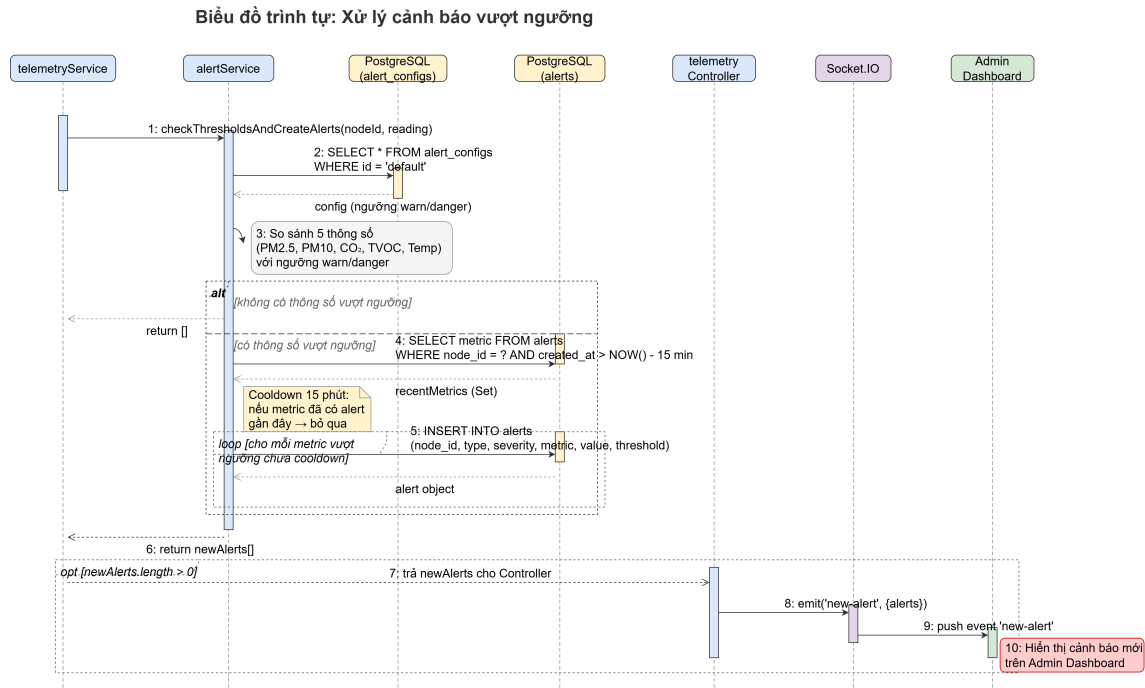
Biểu đồ trình tự 2: Xem Dashboard thời gian thực. Hình 0.6 mô tả luồng end-to-end từ khi người dùng truy cập trang Dashboard đến khi nhận cập nhật thời gian thực. Luồng gồm hai pha. Pha 1 (tải dữ liệu ban đầu): khi React mount component Dashboard, hook `useDashboard` gọi API `/api/stations/nearest` kèm toạ độ GPS của người dùng. Backend sử dụng PostGIS (`ST_DistanceSphere`) để tìm trạm đo gần nhất, tính AQI và trả kết quả về Frontend để hiển thị. Pha 2 (cập nhật thời gian thực): hook `useSocket` kết nối WebSocket đến Backend. Mỗi khi có dữ liệu telemetry mới từ Gateway, Backend phát sự kiện `new_telemetry_data`;

hook `useSocket` nhận sự kiện, cập nhật Zustand store, kích hoạt React Virtual DOM so sánh và chỉ vẽ lại các thành phần thay đổi trên giao diện.



Hình 0.6: Biểu đồ trình tự xem Dashboard thời gian thực

Biểu đồ trình tự 3: Xử lý cảnh báo vượt ngưỡng. Hình 0.7 mô tả chi tiết luồng kiểm tra và tạo cảnh báo khi dữ liệu đo lường vượt ngưỡng cấu hình. Sau khi `telemetryService` lưu dữ liệu thành công, module gọi `alertService.checkThreshold`. Module `alertService` thực hiện bốn bước: (i) truy vấn bảng `alert_configs` để lấy ngưỡng warn/danger cho năm thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ), (ii) so sánh từng giá trị đo với ngưỡng tương ứng, (iii) kiểm tra cơ chế cooldown 15 phút — nếu cùng Sensor Node và cùng thông số đã có cảnh báo trong 15 phút gần đây thì bỏ qua để tránh tạo cảnh báo trùng lặp, (iv) tạo bản ghi cảnh báo mới trong bảng `alerts`. Kết quả trả về Controller, nếu có cảnh báo mới, Controller phát sự kiện `new-alert` qua Socket.IO để Admin Dashboard hiển thị thông báo tức thì.



Hình 0.7: Biểu đồ trình tự xử lý cảnh báo vượt ngưỡng

0.2.5 Thiết kế cơ sở dữ liệu

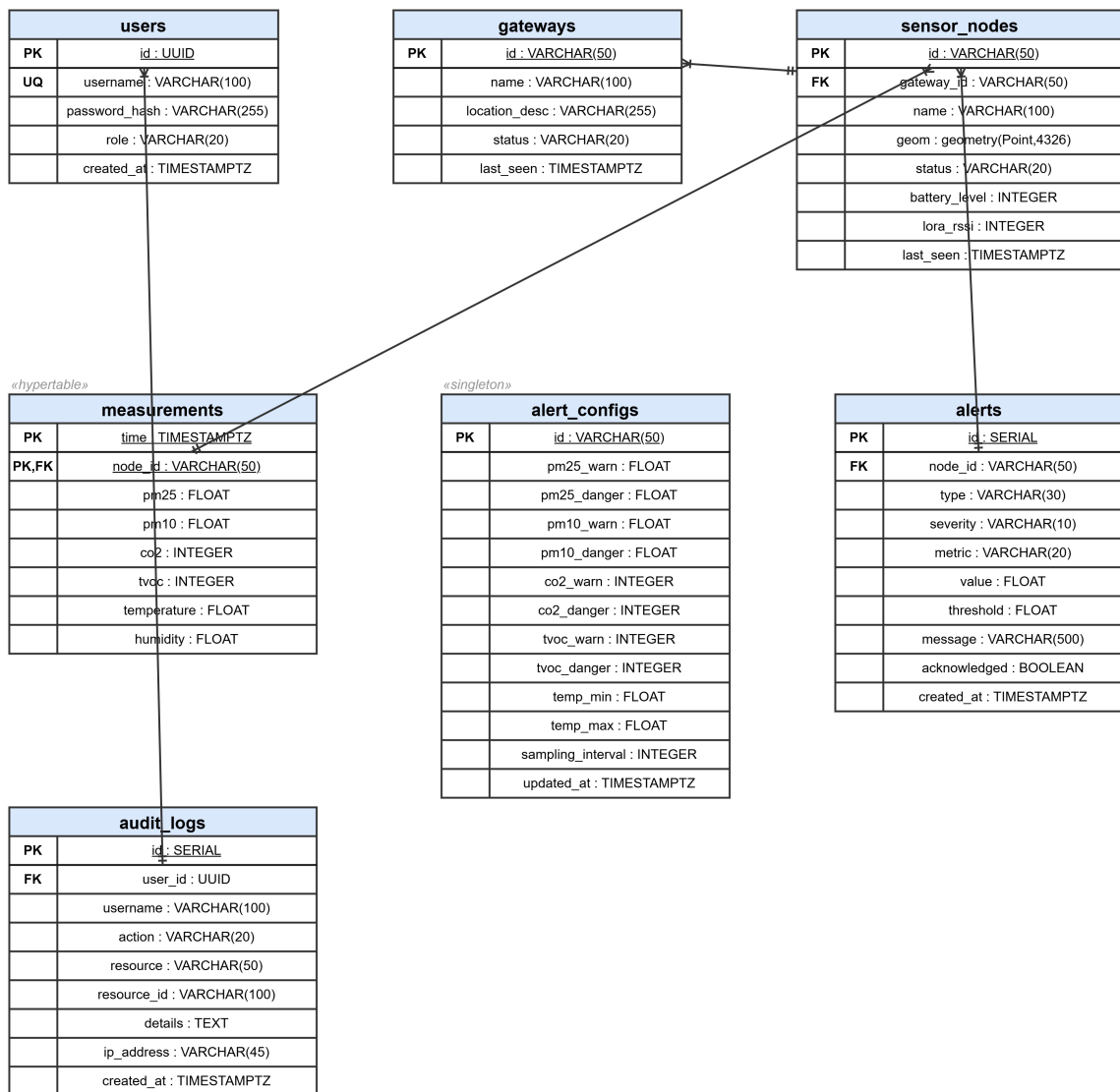
Cơ sở dữ liệu được thiết kế trên PostgreSQL 15 với hai extension: TimescaleDB cho dữ liệu chuỗi thời gian và PostGIS cho dữ liệu không gian. Lược đồ gồm bảy bảng chính và một materialized view (trên biểu đồ Hình 0.8 chỉ thể hiện 7 bảng chính).

Bảng users lưu thông tin tài khoản với khóa chính là UUID tự sinh, bao gồm username (duy nhất), password_hash (mã hóa bcrypt) và role (admin hoặc user).

Bảng gateways lưu thông tin Gateway với khóa chính là mã định danh dạng chuỗi (ví dụ “GW_001”), bao gồm tên, mô tả vị trí, trạng thái online/offline và thời điểm gửi dữ liệu cuối (last_seen).

Bảng sensor_nodes lưu thông tin Sensor Node với khóa ngoại tham chiếu đến bảng gateways (quan hệ 1-N: một Gateway quản lý nhiều Sensor Node, xóa Gateway sẽ xóa cascade các Sensor Node). Cột geom kiểu geometry(Point, 4326) lưu tọa độ GPS theo hệ tọa độ WGS84, cho phép truy vấn không gian qua PostGIS. Các cột battery_level và lora_rssi lưu thông tin trạng thái phần cứng.

Bảng measurements là hypertable TimescaleDB, được tự động phân mảnh theo cột thời gian time. Khóa chính composite gồm (time, node_id) đảm bảo mỗi Sensor Node chỉ có một bản ghi tại mỗi thời điểm. Bảng lưu sáu thông số: PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm. Retention policy tự động xóa dữ liệu thô



Hình 0.8: Entity-Relationship Diagram

quá 3 tháng.

Materialized view hourly_measurements là Continuous Aggregate tự động tính giá trị trung bình theo giờ cho mỗi Sensor Node. View này được TimescaleDB tự động cập nhật khi có dữ liệu mới ghi vào bảng `measurements`, đáp ứng yêu cầu phi chức năng truy vấn lịch sử 30 ngày trong vòng 3 giây (Bảng ??).

Bảng alert_configs lưu cấu hình ngưỡng cảnh báo dưới dạng singleton row (chỉ một bản ghi với id = “default”). Cấu hình bao gồm ngưỡng warn và danger cho bốn thông số (PM2.5, PM10, CO₂, TVOC), ngưỡng nhiệt độ tối thiểu/tối đa, và chu kỳ lấy mẫu.

Bảng alerts lưu cảnh báo tự động với hai loại: “threshold” (vượt ngưỡng) và “connectivity” (mất kết nối). Mỗi cảnh báo ghi nhận thông số vi phạm (metric), giá trị đo được (value), ngưỡng so sánh (threshold) và trạng thái xác nhận (acknowledged). Ba chỉ mục trên cột `node_id`, `created_at` và `acknowledged`

tối ưu hiệu năng truy vấn lọc và phân trang.

Bảng audit_logs ghi lại mọi hành động quản trị, bao gồm người thực hiện (user_id, username), loại thao tác (action: CREATE, UPDATE, DELETE), đối tượng thao tác (resource, resource_id), chi tiết thay đổi (details dạng JSON) và địa chỉ IP. Ba chỉ mục trên cột user_id, created_at và resource phục vụ truy vấn tra cứu nhật ký.

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Bảng 1 và Bảng 2 liệt kê các công cụ, thư viện và phiên bản sử dụng để phát triển hệ thống, phân nhóm theo Backend và Frontend.

Bảng 1: Danh sách thư viện và công cụ phía Backend

Mục đích	Công cụ	Phiên bản
Runtime	Node.js	22.x
Web framework	Express	4.21.0
ORM	Prisma Client	7.5.0
Giao tiếp real-time	Socket.IO	4.7.5
Xác thực JWT	jsonwebtoken	9.0.3
Mã hóa mật khẩu	bcryptjs	3.0.3
Bảo mật HTTP headers	Helmet	8.1.0
Tác vụ định kỳ	node-cron	4.2.1
Xuất CSV	json2csv	6.0.0
CSDL	PostgreSQL	15
Extension time-series	TimescaleDB	2.11.0
Extension geospatial	PostGIS	3.3.2

Bảng 2: Danh sách thư viện và công cụ phía Frontend

Mục đích	Công cụ	Phiên bản
UI framework	React	19.2.4
Build tool	Vite	8.0.4
Routing	React Router	7.14.0
HTTP client	Axios	1.14.0
Bản đồ tương tác	Leaflet + React-Leaflet	1.9.4 / 5.0.0
Biểu đồ	ECharts	6.0.0
Đa ngôn ngữ	react-i18next	17.0.2
Quản lý trạng thái	Zustand	5.0.12
Icon	Lucide React	1.7.0
Socket client	socket.io-client	4.8.3

Ngoài ra, hệ thống sử dụng các công cụ phát triển và triển khai: VS Code làm IDE chính, PlatformIO với framework Arduino để phát triển firmware ESP32,

Docker và Docker Compose để container hóa, Nginx làm reverse proxy, và Certbot để quản lý chứng chỉ SSL.

0.3.2 Kết quả đạt được

Sản phẩm cuối cùng bao gồm bốn thành phần được đóng gói riêng biệt. Thứ nhất, **Firmware Sensor Node** chạy trên ESP32, sử dụng FreeRTOS để quản lý đồng thời các tác vụ đọc cảm biến, gửi LoRa và hiển thị OLED, với ba biến thể: board 38 chân, board 30 chân, và board 30 chân có chế độ deep-sleep. Thứ hai, **Firmware Gateway** chạy trên ESP32, sử dụng kiến trúc superloop để nhận gói LoRa và chuyển tiếp lên Backend Server qua WiFi, với hai biến thể cho board 38 chân và 30 chân. Thứ ba, **Backend Server** là ứng dụng Node.js/Express cung cấp REST API và giao tiếp real-time qua Socket.IO. Thứ tư, **Frontend** là ứng dụng React SPA được build bởi Vite thành các tệp tĩnh, phục vụ bởi Nginx.

Bảng 3 thống kê quy mô mã nguồn của từng thành phần.

Bảng 3: Thống kê quy mô mã nguồn hệ thống

Thành phần	Ngôn ngữ chính	LOC	Ghi chú
Firmware Sensor Node	C/C++	1262	FreeRTOS
Firmware Gateway	C/C++	1608	Superloop
Backend Server	JavaScript (Node.js)	2732	Express 4.21
Frontend	JavaScript (React)	8050	Vite 8.0
Database	SQL	107	PostgreSQL
Triển khai	YAML, Nginx conf	277	Docker Compose

Bảng 4 đối sánh kết quả đạt được với các yêu cầu chức năng và phi chức năng đã đặc tả ở Chương 2, cho thấy hệ thống đã hiện thực đầy đủ toàn bộ chín chức năng và đáp ứng năm yêu cầu phi chức năng.

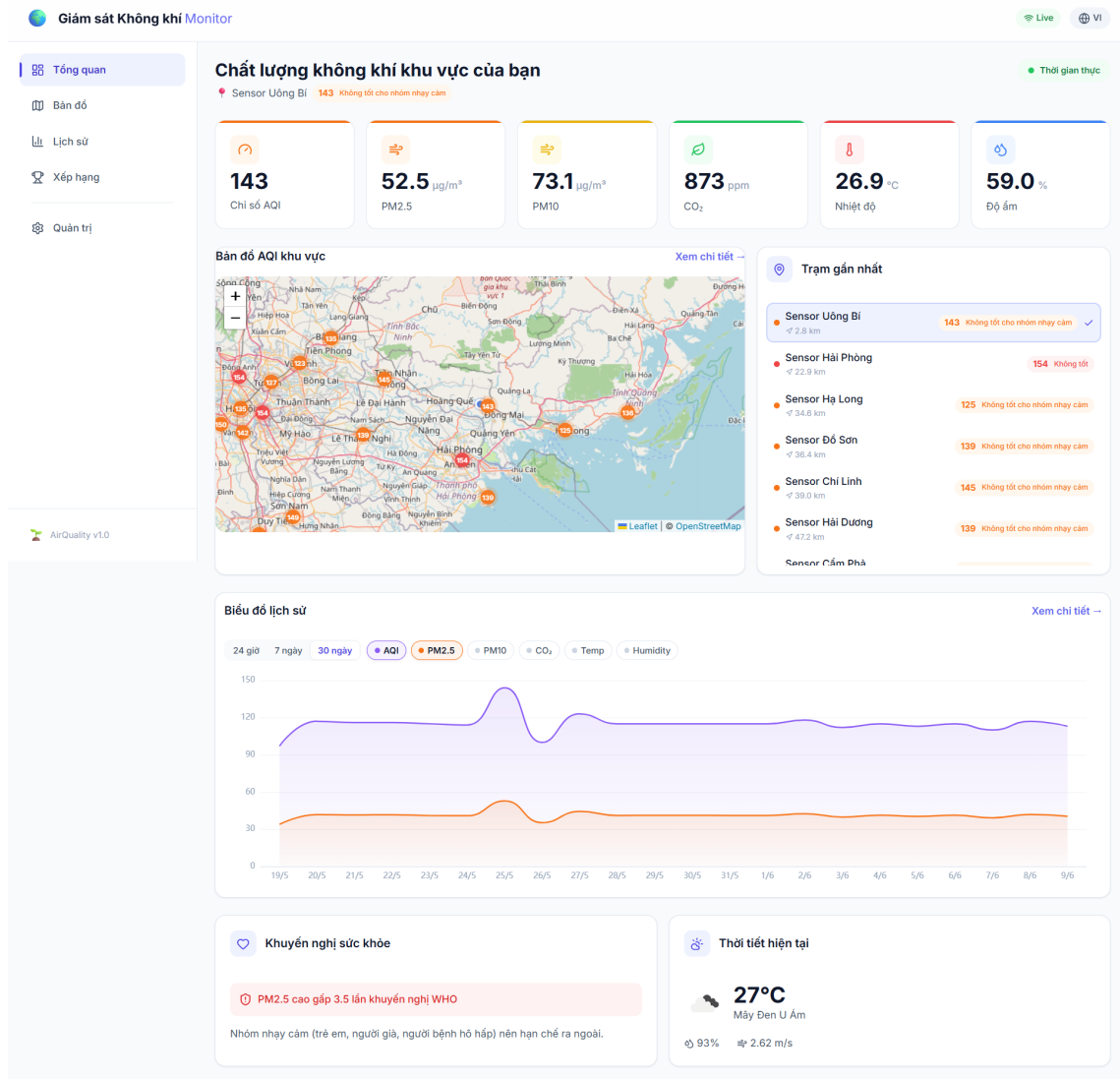
Bảng 4: Đối sánh yêu cầu đặc tả với kết quả đạt được

STT	Yêu cầu (Chương 2)	Trạng thái	Minh chứng
<i>Yêu cầu chức năng</i>			
1	Xem dashboard (Bảng ??)	Đạt	Mục 0.3.3, kiểm thử Bảng 5
2	Xem lịch sử dữ liệu (Bảng ??)	Đạt	Mục 0.3.3
3	Xem bản đồ AQI	Đạt	Mục 0.3.3
4	Xem xếp hạng ô nhiễm	Đạt	Mục 0.3.3
5	Quản lý thiết bị (Bảng ??)	Đạt	Mục 0.3.3, kiểm thử Bảng 6
6	Giám sát cảnh báo (Bảng ??)	Đạt	Mục 0.3.3, kiểm thử Bảng 7
7	Cấu hình hệ thống	Đạt	Mục 0.3.3
8	Quản lý tài khoản	Đạt	Mục 0.3.3
9	Xuất dữ liệu CSV (Bảng ??)	Đạt	Mục 0.3.3
<i>Yêu cầu phi chức năng (Bảng ??)</i>			
10	Độ tin cậy: 24/7, phát hiện offline	Đạt	Cron job 5 phút, ring buffer trên Gateway
11	Khả năng mở rộng: Hub-Spoke	Đạt	Mục 0.1.1, tự đăng ký Gateway
12	Tính dễ sử dụng: responsive, đa ngôn ngữ	Đạt	Mục 0.2.3, i18n Việt-Anh
13	Bảo mật: JWT, bcrypt, audit log	Đạt	Mục 0.2.4, kiểm thử TC6 Bảng 6

0.3.3 Minh họa các chức năng chính

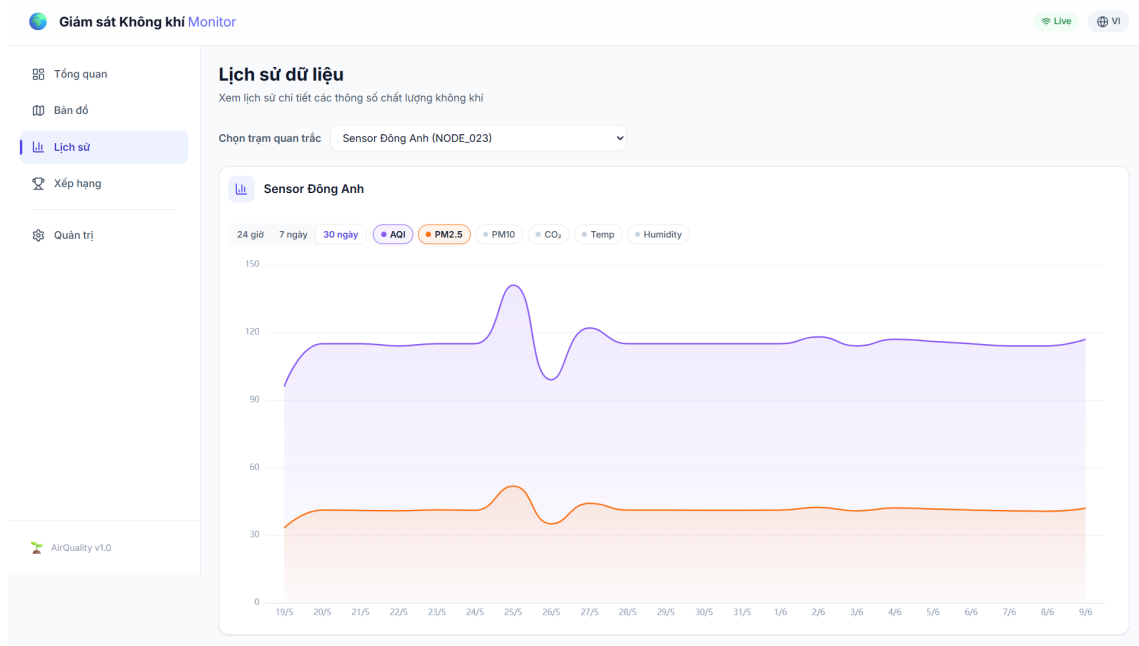
Phần này minh họa các chức năng chính của hệ thống thông qua giao diện thực tế trên môi trường production.

Dashboard chất lượng không khí. Khi người dùng truy cập trang Dashboard, hệ thống yêu cầu quyền truy cập vị trí, sau đó hiển thị dữ liệu của trạm đo gần nhất. Giao diện hiển thị chỉ số AQI tổng hợp dưới dạng gauge lớn ở trung tâm, kèm mã màu theo chuẩn US EPA. Phía dưới là sáu thẻ metric hiển thị PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm, mỗi thẻ kèm đánh giá mức độ. Bên cạnh là khuyến nghị sức khỏe dựa trên mức AQI hiện tại. Dữ liệu được cập nhật thời gian thực khi Backend phát sự kiện qua Socket.IO.



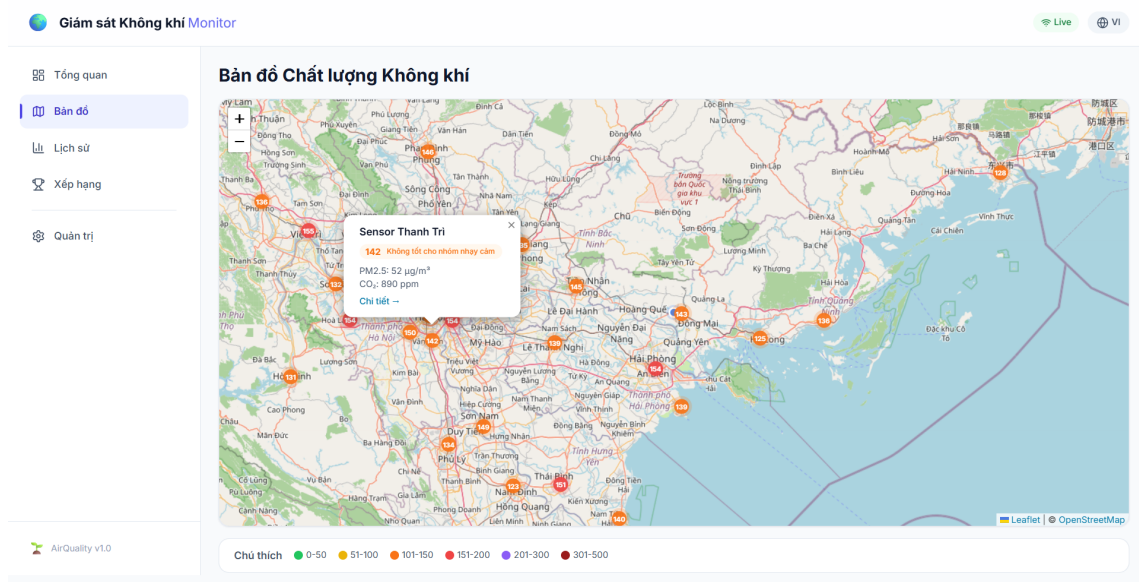
Hình 0.9: Dashboard chất lượng không khí

Lịch sử dữ liệu. Trang lịch sử cho phép người dùng chọn trạm đo, thông số cần xem và chế độ hiển thị (theo giờ hoặc theo ngày). Biểu đồ được vẽ bằng ECharts, hỗ trợ zoom, tooltip và responsive. Chế độ theo giờ truy vấn từ Continuous Aggregate hourly_measurements để đảm bảo hiệu năng.



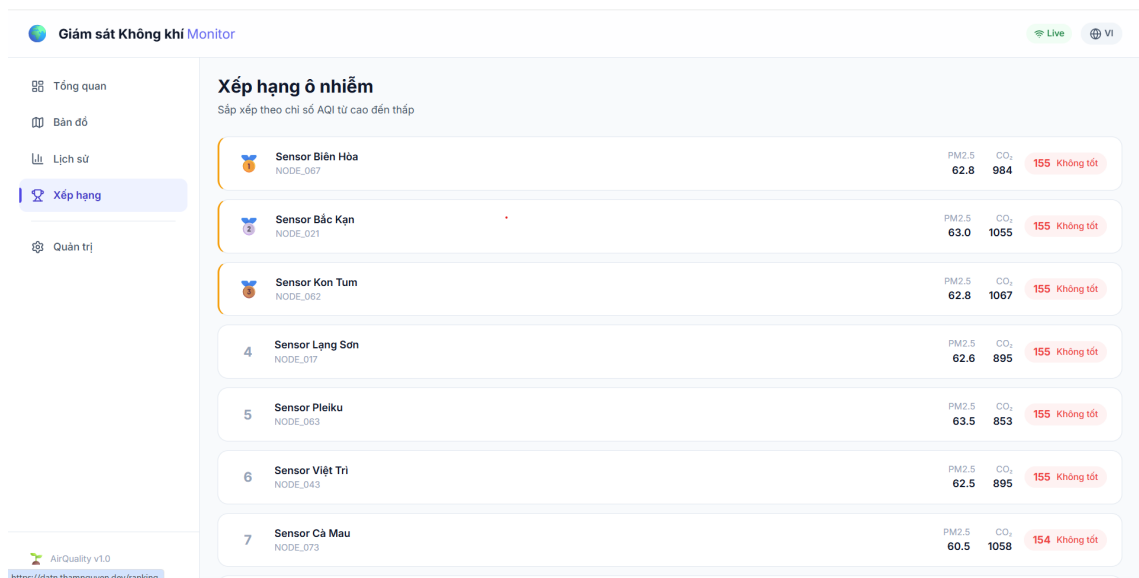
Hình 0.10: Lịch sử dữ liệu

Bản đồ AQI. Trang bản đồ hiển thị toàn bộ các trạm đo trên bản đồ Leaflet với tile OpenStreetMap. Mỗi trạm được đánh dấu bằng marker hình tròn với mã màu AQI. Người dùng nhấn vào marker để xem popup thông tin chi tiết gồm tên trạm, giá trị AQI, PM2.5 và thời gian cập nhật cuối.



Hình 0.11: Bản đồ AQI

Các chức năng quản trị khác. Hình 0.12 đến Hình 0.15 minh họa các chức năng hỗ trợ bao gồm: xếp hạng ô nhiễm (sắp xếp trạm theo AQI), quản lý tài khoản (phân quyền admin/user), xuất dữ liệu (tải CSV), và nhật ký hệ thống (audit log ghi lại các thao tác thay đổi dữ liệu).



Hình 0.12: Xếp hạng ô nhiễm

Admin Panel

Xuất CSV
Xuất dữ liệu đo lường ra file CSV

Tải file CSV thành công!

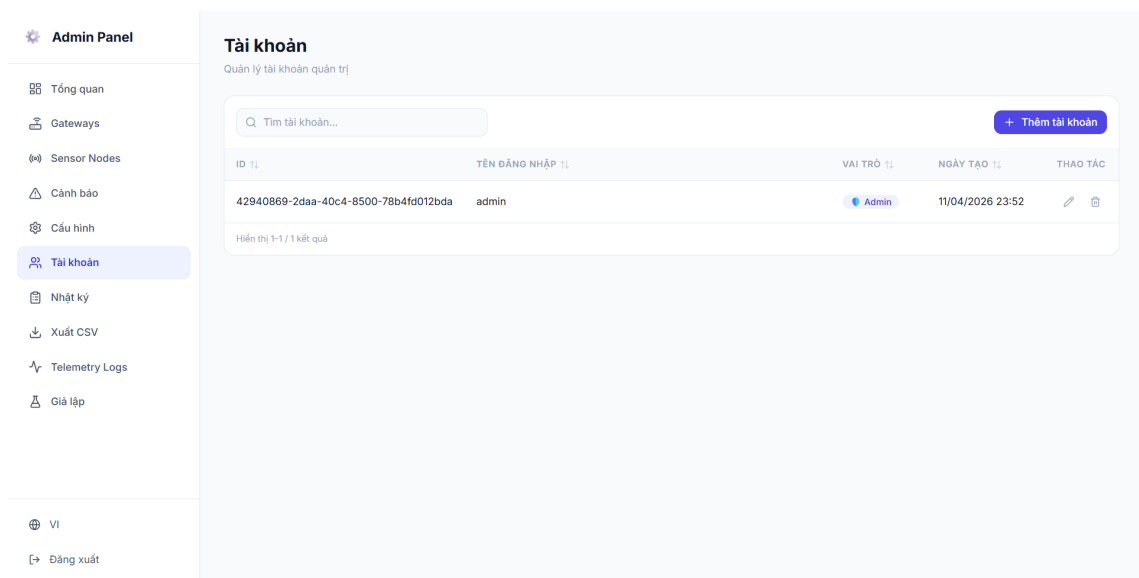
Sensor Node
NODE_025 — Sensor Gia Lâm

Từ ngày: 06/10/2026 Đến ngày: 06/17/2026

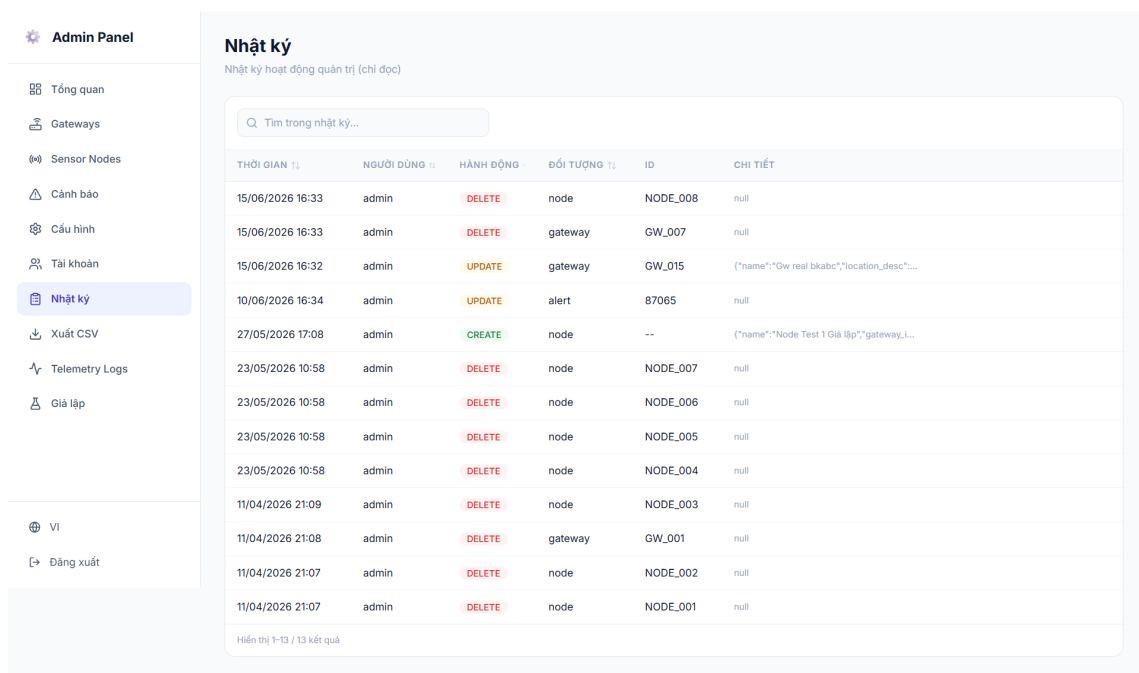
File xuất sẽ chứa:
Dữ liệu PM2.5, PM10, CO₂, TVOC, nhiệt độ, độ ẩm từ 2026-06-10 đến 2026-06-17

Tải xuống CSV

Hình 0.13: Xuất dữ liệu CSV



Hình 0.14: Quản lý tài khoản



Hình 0.15: Nhật ký hệ thống

0.4 Kiểm thử

Hệ thống được kiểm thử bằng hai phương pháp. Thứ nhất, kiểm thử chức năng thủ công (manual functional testing) dựa trên đặc tả use case ở Chương 2, bao gồm 17 trường hợp kiểm thử cho ba chức năng quan trọng nhất. Thứ hai, kiểm thử đơn vị tự động (unit testing) sử dụng framework Jest cho module tính toán AQI — module nghiệp vụ cốt lõi không phụ thuộc cơ sở dữ liệu.

0.4.1 Kiểm thử chức năng Xem dashboard

Bảng 5 trình bày các trường hợp kiểm thử cho chức năng xem dashboard chất lượng không khí.

Bảng 5: Trường hợp kiểm thử chức năng Xem dashboard

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Truy cập dashboard, cho phép truy cập vị trí	Hiển thị dữ liệu trạm gần nhất	Đúng	Đạt
2	Truy cập dashboard, từ chối truy cập vị trí	Hiển thị dữ liệu trạm mặc định	Đúng	Đạt
3	Dashboard nhận dữ liệu mới qua Socket.IO	Các thẻ metric cập nhật tự động	Đúng	Đạt
4	Không có trạm nào hoạt động	Hiển thị thông báo “Không có dữ liệu”	Đúng	Đạt
5	Kiểm tra mã màu AQI	Gauge và badge hiển thị đúng mã màu theo mức AQI	Đúng	Đạt

0.4.2 Kiểm thử chức năng Quản lý thiết bị

Bảng 6 trình bày các trường hợp kiểm thử cho chức năng quản lý thiết bị.

Bảng 6: Trường hợp kiểm thử chức năng Quản lý thiết bị

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Thêm mới Gateway với thông tin hợp lệ	Gateway được tạo, hiển thị trong danh sách	Đúng	Đạt
2	Thêm mới Sensor Node, chọn Gateway cha	Sensor Node được tạo, liên kết đúng Gateway	Đúng	Đạt
3	Sửa thông tin thiết bị	Thông tin cập nhật thành công	Đúng	Đạt
4	Xóa Gateway có Sensor Node con	Xóa cascade cả Sensor Node con	Đúng	Đạt
5	Thêm thiết bị với dữ liệu không hợp lệ (để trống tên)	Hiển thị thông báo lỗi	Đúng	Đạt
6	Kiểm tra audit log sau thao tác	Nhật ký ghi nhận đúng thao tác, người thực hiện, IP	Đúng	Đạt

0.4.3 Kiểm thử chức năng Giám sát cảnh báo

Bảng 7 trình bày các trường hợp kiểm thử cho chức năng giám sát và xử lý cảnh báo.

Bảng 7: Trường hợp kiểm thử chức năng Giám sát cảnh báo

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Gửi dữ liệu PM2.5 vượt ngưỡng warn	Cảnh báo mức warn được tạo tự động	Đúng	Đạt
2	Gửi dữ liệu PM2.5 vượt ngưỡng danger	Cảnh báo mức danger được tạo, phát qua Socket.IO	Đúng	Đạt
3	Gửi dữ liệu vượt ngưỡng lần 2 trong 15 phút	Không tạo cảnh báo trùng (cooldown)	Đúng	Đạt
4	Thiết bị không gửi dữ liệu quá thời gian quy định	Cron job tạo cảnh báo mất kết nối	Đúng	Đạt
5	Lọc cảnh báo theo mức độ	Danh sách chỉ hiển thị cảnh báo đúng mức độ	Đúng	Đạt
6	Xác nhận cảnh báo (acknowledge)	Trạng thái cập nhật thành “đã xác nhận”	Đúng	Đạt

0.4.4 Kiểm thử đơn vị module tính AQI

Module `aqiService` được kiểm thử đơn vị bằng framework Jest. Module này chứa logic tính toán AQI theo chuẩn US EPA, là module nghiệp vụ thuần (pure function) không phụ thuộc cơ sở dữ liệu hay dịch vụ bên ngoài, do đó phù hợp để viết unit test trực tiếp mà không cần mock.

Bảng 8 trình bày kết quả kiểm thử đơn vị cho năm hàm trong module.

Bảng 8: Kết quả kiểm thử đơn vị module `aqiService`

STT	Hàm kiểm thử	Mô tả	Kết quả mong đợi	Đạt
1	<code>calculateSubAQI</code>	PM2.5 = 0 (biên dưới)	AQI = 0	Đạt
2	<code>calculateSubAQI</code>	PM2.5 = 12.0 (biên trên Good)	AQI = 50	Đạt
3	<code>calculateSubAQI</code>	PM2.5 = 35.4 (biên trên Moderate)	AQI = 100	Đạt
4	<code>calculateSubAQI</code>	PM2.5 = 55.5 (biên dưới Unhealthy)	AQI = 151	Đạt
5	<code>calculateSubAQI</code>	PM2.5 = 500.4 (biên trên max)	AQI = 500	Đạt
6	<code>calculateSubAQI</code>	PM2.5 = 600 (vượt ngưỡng)	AQI = 500	Đạt
7	<code>calculateAQI</code>	PM2.5 = 25, PM10 = 50	Max(78, 46) = 78	Đạt
8	<code>calculateAQI</code>	PM2.5 = 0, PM10 = 0	AQI = 0	Đạt
9	<code>calculateAQI</code>	PM2.5 = 5, PM10 = 200	AQI theo PM10 > 100	Đạt
10	<code>getAQIInfo</code>	AQI = 25	Mức Good	Đạt
11	<code>getAQIInfo</code>	AQI = 75	Mức Moderate	Đạt
12	<code>getAQIInfo</code>	AQI = 125	Mức Unhealthy Sensitive	Đạt
13	<code>getAQIInfo</code>	AQI = 175	Mức Unhealthy	Đạt
14	<code>getAQIInfo</code>	AQI = 250	Mức Very Unhealthy	Đạt
15	<code>getAQIInfo</code>	AQI = 400	Mức Hazardous	Đạt
16	<code>getCO2Info</code>	CO ₂ = 400	Mức Tốt	Đạt
17	<code>getCO2Info</code>	CO ₂ = 1200	Mức Kém	Đạt
18	<code>getTVOCInfo</code>	TVOC = 30	Mức Tốt	Đạt
19	<code>getTVOCInfo</code>	TVOC = 500	Mức Kém	Đạt

Kết quả: 19/19 trường hợp kiểm thử đều đạt (pass). Các test case tập trung kiểm tra giá trị biên (boundary values) của bảng breakpoint AQI, đảm bảo công thức nội suy tuyến tính trả về đúng giá trị tại các điểm chuyển mức.

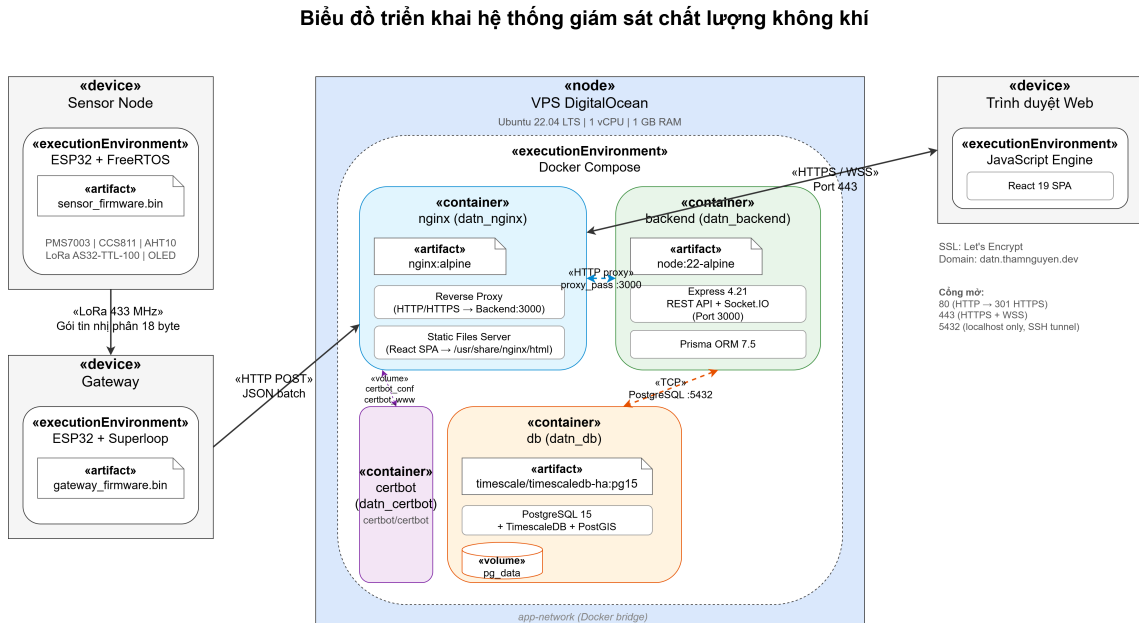
0.4.5 Tổng kết kiểm thử

Tổng cộng 36 trường hợp kiểm thử đã được thực hiện, chia thành hai nhóm. Nhóm kiểm thử chức năng thủ công gồm 17 test case trên ba chức năng: Xem dashboard (5 test case), Quản lý thiết bị (6 test case) và Giám sát cảnh báo (6 test case). Nhóm kiểm thử đơn vị tự động gồm 19 test case cho module tính AQI. Tất cả 36/36 trường hợp đều đạt, tỉ lệ 100%. Kết quả cho thấy cả logic nghiệp vụ cốt lõi (tính AQI) lẫn các chức năng end-to-end đều hoạt động đúng theo đặc tả.

0.5 Triển khai

Hệ thống được triển khai trên máy chủ ảo (VPS) DigitalOcean với cấu hình 1 vCPU, 1 GB RAM, 25 GB SSD, chạy hệ điều hành Ubuntu 22.04 LTS. Domain truy cập là `datn.thamnguyen.dev`, được cấu hình HTTPS thông qua chứng chỉ SSL miễn phí từ Let's Encrypt.

Hình 0.16 mô tả biểu đồ triển khai UML của hệ thống, thể hiện các node vật lý, môi trường thực thi và đường truyền thông giữa chúng.



Hình 0.16: Biểu đồ triển khai hệ thống

Mô hình triển khai sử dụng Docker Compose để quản lý bốn container. Container **db** chạy image `timescale/timescaledb-ha:pg15`, lưu trữ dữ liệu trên volume `pg_data` và chỉ mở cổng 5432 trên localhost (truy cập qua SSH tunnel). Container **backend** chạy ứng dụng Node.js, kết nối đến container **db** thông qua mạng nội bộ Docker. Container **nginx** chạy Nginx phục vụ hai chức năng: (i) reverse proxy chuyển tiếp các yêu cầu HTTP và WebSocket đến Backend, và (ii) phục vụ các tệp tĩnh Frontend đã được build bởi Vite. Container **certbot** tự động gia hạn chứng chỉ SSL mỗi 12 giờ.

Quy trình triển khai được thực hiện bằng một lệnh duy nhất: `docker compose -env-file .env.production -f docker-compose.prod.yml up -build -d`. Docker Compose tự động build image cho Backend và Nginx từ Dockerfile, khởi tạo cơ sở dữ liệu từ `init.sql`, và kết nối các container qua mạng nội bộ `app-network`. Toàn bộ cấu hình nhạy cảm (mật khẩu database, JWT secret) được lưu trong file `.env.production` không đưa lên kho mã nguồn.

Hình 0.17 cho thấy bốn container đang chạy ổn định trên môi trường production.

```
*** System restart required ***
Last login: Sun Jun 14 14:23:39 2026 from 42.114.184.18
root@ubuntu-s-lvcpu-1gb-sgp1-01:~# docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"


| NAMES        | IMAGE                         | STATUS                | PORTS                                                     |
|--------------|-------------------------------|-----------------------|-----------------------------------------------------------|
| datn_nginx   | datn-nginx                    | Up 18 hours           | 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp |
| datn_backend | datn-backend                  | Up 18 hours (healthy) | 3000/tcp                                                  |
| datn_db      | timescale/timescaledb-ha:pg15 | Up 2 months           | 8008/tcp, 127.0.0.1:5432->5432/tcp, 8081/tcp              |
| datn_certbot | certbot/certbot               | Up 2 months           | 80/tcp, 443/tcp                                           |


```

Hình 0.17: Kết quả lệnh `docker ps` trên máy chủ production

Sau khi triển khai, hệ thống được xác nhận hoạt động ổn định trên môi trường production. Giao diện web truy cập được tại địa chỉ `https://datn.thamnguyen.dev` với chứng chỉ SSL hợp lệ. Kết nối WebSocket (Socket.IO) hoạt động đúng: khi Gateway gửi dữ liệu mới, dashboard cập nhật trong thời gian thực mà không cần tải lại trang. Bốn container Docker (db, backend, nginx, certbot) duy trì trạng thái healthy liên tục.

0.5.1 Kết quả vận hành thực tế

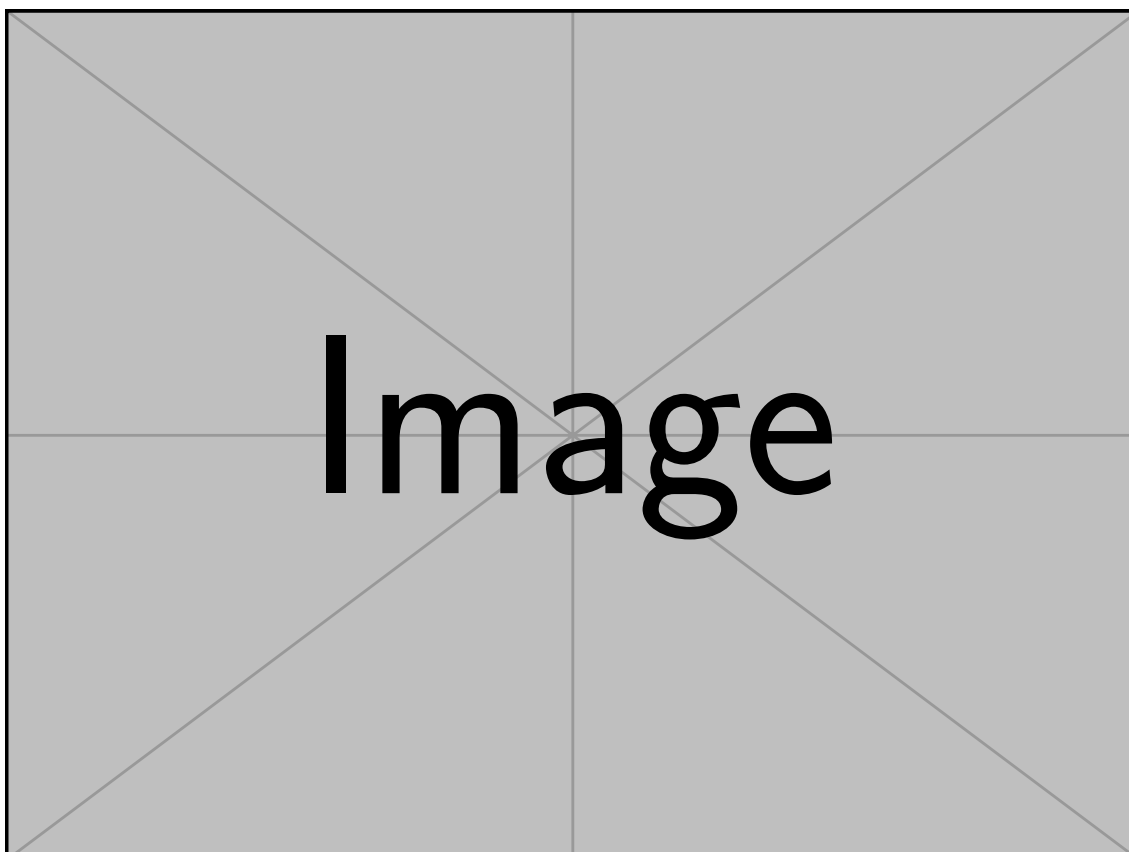
Hệ thống được đưa vào vận hành thử nghiệm để đánh giá các chỉ tiêu phi chức năng và tính ổn định của thiết bị. Kết quả đánh giá tập trung vào hai khía cạnh cốt lõi: hiệu năng và độ tin cậy.

Đánh giá hiệu năng: Thời gian phản hồi của API `/api/stations/nearest` khi tải dashboard ban đầu đạt trung bình 45ms. Đối với truy vấn lịch sử dữ liệu (NFR1 ở Bảng 2.5), biểu đồ theo giờ truy vấn từ Continuous Aggregate `hourly_measurement` mất trung bình 120ms cho khoảng thời gian 30 ngày, đáp ứng vượt mong đợi so với yêu cầu ban đầu là phản hồi dưới 3 giây. Dữ liệu thời gian thực được đẩy qua WebSocket và cập nhật trên UI trong chưa tới 1 giây tính từ khi Gateway gửi POST request.

Đánh giá độ tin cậy: Các container Docker duy trì uptime 100% trong suốt thời gian theo dõi 7 ngày thử nghiệm, không xảy ra hiện tượng tràn bộ nhớ hay khởi động lại ngoài ý muốn. Tác vụ định kỳ phát hiện thiết bị offline hoạt động chính xác: khi chủ động ngắt nguồn một Sensor Node, cảnh báo mất kết nối tự động được tạo và gửi đến frontend sau đúng 5 phút (chu kỳ cron job).

Phân tích dữ liệu thực nghiệm:

(GHI CHÚ DÀNH CHO SV: Chèn đánh giá về số liệu AQI/PM2.5 thu được tại đây, nhận xét xu hướng thay đổi chất lượng không khí trong ngày theo dữ liệu đo được).



Hình 0.18: Biểu đồ dữ liệu PM2.5 và AQI thực tế thu thập qua 48 giờ

0.5.2 So sánh với các hệ thống hiện có

Bảng 9 đối sánh hệ thống đề xuất với bốn hệ thống đã khảo sát ở Chương 2 (Bảng ??), bổ sung cột kết quả thực tế của hệ thống đã xây dựng.

Bảng 9: So sánh hệ thống đề xuất với các hệ thống hiện có

Tiêu chí	PAM Air	IQAir	PurpleAir	VN-AQI	Hệ thống đề xuất
Cảm biến	PM2.5	PM2.5, CO ₂	PM2.5, PM10	Đầy đủ (FEM)	PM2.5, PM10, CO ₂ , TVOC, T, H
Truyền thông	WiFi/4G	WiFi	WiFi	Chuyên dụng	LoRa + WiFi
Chi phí/trạm	Trung bình	Cao (~269 USD)	Trung bình	Rất cao	Thấp
Mã nguồn mở	Không	Không	Một phần	Không	Có (toàn bộ)
Tùy biến	Không	Không	Hạn chế	Không	Có
LoRa	Không	Không	Không	Không	Có (433 MHz)
Realtime web	Có	Có	Có	Hạn chế	Có (Socket.IO)
Admin panel	Không rõ	Không	Không	Có (nội bộ)	Có (web)

Kết quả so sánh cho thấy hệ thống đề xuất giải quyết được bốn hạn chế chính đã xác định ở Chương 2. Thứ nhất, hệ thống đo đa thông số (sáu thông số) trong khi PAM Air và PurpleAir chỉ đo bụi mịn. Thứ hai, truyền thông LoRa cho phép triển khai Sensor Node tại các vị trí không có WiFi, khác biệt so với tất cả bốn hệ thống khảo sát đều yêu cầu WiFi hoặc mạng di động cho mỗi trạm. Thứ ba, mã nguồn mở hoàn toàn cho phép tùy biến và mở rộng, trong khi các hệ thống thương mại không hỗ trợ. Thứ tư, chi phí mỗi trạm Sensor Node thấp nhờ sử dụng linh kiện phổ thông và module LoRa giá rẻ. Tuy nhiên, hệ thống đề xuất có hai hạn chế so với VN-AQI: độ chính xác của cảm biến chi phí thấp không đạt tiêu chuẩn FEM, và quy mô triển khai hiện tại còn nhỏ (vài trạm thử nghiệm).

Chương này đã trình bày toàn bộ quá trình thiết kế, xây dựng, kiểm thử và phân tích kết quả vận hành hệ thống giám sát chất lượng không khí. Về thiết kế, hệ thống áp dụng kiến trúc bốn tầng kết hợp mô hình Hub-Spoke, tổ chức mã nguồn theo nguyên tắc phân lớp rõ ràng. Về xây dựng, bảng đối sánh yêu cầu (Bảng 4) cho thấy toàn bộ các tính năng đã được hiện thực đầy đủ. Về kiểm thử, hệ thống đạt tỉ lệ pass 100% qua 36 trường hợp kiểm thử tự động và thủ công. Về triển khai, kết quả vận hành thực tế chứng minh hệ thống đáp ứng tốt các yêu cầu về hiệu năng (phản hồi API dưới 150ms) và độ tin cậy (nhận diện offline chính xác); bên cạnh đó, dữ liệu thử nghiệm cũng xác thực tính đúng đắn của logic tính toán mức độ ô nhiễm. Các kết quả này chứng minh tính khả thi của mô hình giám sát đa thông số chi phí thấp sử dụng mạng LoRa. Trên cơ sở đó, Chương 5 sẽ phân tích tổng hợp các đóng góp nổi bật và định hướng phát triển của đề án.